Intune Managed App Configuration Policies (iOS & Android) — Zero-Touch App Setup & SSO Routing

Implementation Solution Guide

Version 1.0.0 · 2026-07-30

Contents

Executive Summary — Intune Managed App Configuration Policies (iOS & Android) 4

1. Solution Overview — Intune Managed App Configuration Policies (iOS & Android) 4

What App Configuration Policies are . . . 4

The two delivery channels — and when each applies . . . 5

Authoring the key/value pairs . . . 6

Identity and SSO routing (high level) . . . 6

The worked example — SAP Ariba Procurement . . . 7

How app config interacts with App Protection and supersedence . . . 7

Benefits . . . 7

Limitations (be honest with buyers) . . . 7

The standard deployment model . . . 8

2. Requirements 8

2.1 Microsoft Intune plan and Entra ID licensing . . . 8

2.2 Enrollment posture matched to the delivery channel . . . 8

2.3 Apple MDM push certificate (iOS/iPadOS managed-device channel) . . . 9

2.4 Managed Google Play connection (Android Enterprise managed-device channel) . . . 9

2.5 The target app added to Intune, identified by exact publisher + bundle/package ID . . . 10

2.6 The vendor's supported configuration keys . . . 10

2.7 Microsoft Entra single sign-on plumbing (for SSO routing) . . . 10

2.8 A Conditional Access policy scoped to the app (enforcement — deferred) . . . 11

2.9 A pilot Entra security group . . . 11

2.10 An admin workstation with Microsoft Graph PowerShell (for the included scripts) . . . 11

2.11 US Government (GCC / GCC High / DoD) endpoints — if applicable . . . 12

Global Requirements — Intune Solutions (category-level) 12

G1. Microsoft Entra ID tenant . . . 12

G2. Microsoft Intune subscription . . . 13

G3. Administrative roles . . . 13

G4. Device enrollment in Intune . . . 13

G5. Pilot security group . . . 13

G6. Supported Windows baseline . . . 13

G7. Network access to Microsoft cloud endpoints . . . 13

3. Implementation 13

Phase A — Assess and choose the channel . . . 14

Phase B — Build the iOS/iPadOS App Configuration Policy (Ariba, managed devices) . . . 15

Phase C — Wire sign-in to Entra ID / Conditional Access . . . 17

Phase D — Build the Android Enterprise managed configuration (general pattern) . . . 17
Phase E — Managed-app (MAM) channel — when the device is unenrolled . . . 19
Phase F — Pilot → Validate → Expand . . . 19
5. Testing & Validation 20
The end-to-end acceptance test (the one that matters) . . . 20
Test 1 — Policy targets the device/user . . . 20
Test 2 — App opens pre-configured (the zero-touch gate) . . . 20
Test 3 — Sign-in routes to Entra ID and Conditional Access applies . . . 20
Test 4 — Confirm the delivered values on the device . . . 20
Test 5 — Android managed configuration (if used) . . . 21
Test 6 — Managed-app (MAM) channel (if used) . . . 21
Test 7 — App Protection coexistence (if both are assigned) . . . 21
Test 8 — Reporting coverage . . . 21
Exit criteria to expand from pilot to production . . . 21
6. Troubleshooting 22
Portal / plist errors . . . 22
Config not applying . . . 22
MAM vs MDM confusion . . . 23
SSO / Conditional Access routing . . . 23
Supersedence & coexistence . . . 24
Scripts / Graph . . . 24
7. Best Practices 25
Design . . . 25
Naming & hygiene . . . 25
Identity & SSO . . . 25
Rollout & change control . . . 26
Operations . . . 26
8. Deliverables Summary 26
What you receive . . . 26
Outcome . . . 27
Accuracy note . . . 27
Appendix A — PowerShell Scripts 28

Executive Summary — Intune Managed App Configuration Policies (iOS & Android)

Most mobile-app friction is not a licensing problem — it is a *first-launch* problem. A user installs a line-of-business app, then is asked to type a server hostname, a tenant or realm, an environment name, and an account. They guess, they mistype, they call the help desk, and the app never routes cleanly to your identity provider. Multiply that across a fleet and the cost is real.

App Configuration Policies in Microsoft Intune remove that friction. They push the app's own settings — server hostname, tenant/realm, environment, feature flags, and sign-in routing — *to the app* so it configures itself the moment it opens. The user launches the app and lands on the right sign-in screen, already pointed at your Microsoft Entra ID tenant and covered by Conditional Access. Zero-touch setup, fewer tickets, and a consistent, auditable configuration everywhere.

This solution delivers the repeatable way to build those policies across **both platforms**:

- **iOS/iPadOS** — managed-device app configuration authored as a proper Apple **property list (plist)**, plus the managed-app (MAM) channel for unenrolled devices.
- **Android Enterprise** — **managed configurations** read from the app's own schema in managed Google Play, via the configuration designer or the JSON channel.
- **The two delivery channels explained** — *managed devices* (enrolled/MDM) versus *managed apps* (MAM) — so you always pick the channel that will actually reach your users.
- **A full worked example — SAP Ariba Procurement** — an App Configuration Policy that pre-fills the Ariba site/realm/environment, sets the user's role option, and routes sign-in to Entra ID and Conditional Access, built click-by-click in the Intune admin center with every object named.
- **Identity/SSO routing** — how the app trusts Entra single sign-on and how a Conditional Access policy scoped to that app enforces access (the deep CA build is handled by the dedicated Conditional Access solution; here we wire the app to route to it).
- **Five Microsoft Graph PowerShell tools** — read-only by default, cloud-aware (Commercial and US Gov) — to pre-flight scopes, export existing policies and assignments, validate delivery to a device, report configuration coverage, and stage guarded remediation.

Delivered as self-guided, click-by-click documentation and tested scripts — buy once, reuse across unlimited apps and devices. The outcome: apps that set themselves up correctly on first launch and route every sign-in to your identity provider, with the configuration proven on a pilot device before you expand.

1. Solution Overview — Intune Managed App Configuration Policies (iOS & Android)

What App Configuration Policies are

An **App Configuration Policy** in Microsoft Intune pushes an app's own settings — the values the app would otherwise ask a user to type on first launch — directly to the app, so it arrives **pre-configured**. Instead of a user entering a server hostname, a tenant/realm, an environment, or an account, Intune delivers those key/value pairs and the app reads them the moment it opens.

This is deliberately different from other Intune app features:

• **App Configuration** = *settings inside the app* (server hostname, realm, environment, feature flags, sign-in routing). This solution.

• **App Protection (MAM / APP)** = *data-protection controls* (encryption, copy/paste and save-as restrictions, app PIN). A different, complementary feature — covered by the dedicated App Protection solution.

• **App deployment/assignment** = *getting the app onto the device* (required/available install).

App configuration answers one question: **"When this app opens, how is it already set up, and where does it send the user to sign in?"**

The two delivery channels — and when each applies

App Configuration Policies are delivered through one of two channels. Choosing the right one is the single most important design decision, because a policy sent on the wrong channel silently never applies.

1. **Managed devices (the MDM channel)** — for devices **enrolled** in Intune.

• **iOS/iPadOS:** the settings are delivered as an Apple **Managed App Configuration** payload — an Apple **property list (plist)** of key/value pairs the app declares support for. Intune targets the enrolled device, and the OS hands the plist to the app.

• **Android Enterprise:** the settings are delivered as **managed configurations** — the app publishes a configuration **schema** to managed Google Play, and Intune renders that schema so you set each key. Applies to Android Enterprise (work profile, fully managed, dedicated).

• Use this channel when devices are **enrolled** and you want the app configured at the device level for every user of that device.

1. **Managed apps (the MAM channel)** — for **MAM-managed apps**, enrolled **or** unenrolled (BYOD).

• Settings flow through the **Intune App SDK / App Protection** channel to apps that integrate it (the Microsoft 365 apps and other SDK-enabled apps). The policy is scoped to a **user**, and the app reads the configuration when the user signs in with their work account.

• Use this channel when the device is **not enrolled** (personal/BYOD) but you still need to hand the app settings — or when you want configuration tied to the *user's identity* rather than the device.

> ***The enrolled vs MAM distinction in one line:*** **managed-device* app config targets the **device** (enrolled/MDM) and can configure any app that supports the OS payload; *managed-app* app config targets the **user** (MAM) and only works with apps that integrate the Intune App SDK. If you send a managed-device iOS plist to an unenrolled phone, nothing happens — the OS payload channel does not exist there.*

Authoring the key/value pairs

The values you push are always **key/value pairs**, but the format depends on the platform and the authoring surface you choose.

• **iOS/iPadOS — "Configuration designer" vs "Enter XML data".** Intune offers two ways to author an iOS managed-device policy:

• **Configuration designer** — you type the **key** and **value** into a table; Intune builds the payload for you. Use this whenever you can; it is the least error-prone.

• **Enter XML data** — you paste an Apple **property list (plist)** directly. This is **not** a place for a vendor's XML *profile* schema. The content must be a valid plist: a lowercase `<dict>` root containing only `<key>`/`<string>` (and `<integer>`/`<true/>`/etc.) pairs, with **no** XML namespace attributes (`xmlns`, `xsi`) and **no** vendor profile headers. A capitalized `<Dict>` root or a namespaced document is rejected by the portal. If an app genuinely needs a whole configuration document as one setting, it goes in as a **string value under a key** (e.g. a CDATA string), never as the plist's structure.

• **Android Enterprise — the managed configurations schema.** You do not invent keys. The app publishes its supported keys to managed Google Play; Intune reads that **schema** and renders the **configuration designer** so you fill in each key's value. There is no free-typed XML on Android — the schema is authoritative, and keys the app did not publish cannot be set.

• **Where the keys come from — always the vendor.** For any third-party app, the exact key **names** are defined by the app vendor and published in the app's documentation (and, on Android, in the managed-configuration schema). Never guess a key name; a typo'd key is simply ignored by the app.

Identity and SSO routing (high level)

A well-built App Configuration Policy does more than pre-fill a hostname — it **routes the user's sign-in to Microsoft Entra ID** so Conditional Access can enforce access. Two pieces cooperate:

1. **The app trusts Entra single sign-on.** Modern apps authenticate through **MSAL** (the Microsoft Authentication Library) and honor platform SSO — on iOS the **Microsoft Enterprise SSO plug-in** (delivered by the Microsoft Authenticator / Company Portal SSO extension), on Android the broker in Authenticator/Company Portal. App configuration keys point the app at your **tenant** and, where the app supports it, tell it to use federated/SSO sign-in rather than a local username/password.

1. **A Conditional Access policy scoped to the app enforces the access rules.** Once the app routes sign-in to Entra ID, a CA policy targeting that cloud app applies your controls (require MFA, require a compliant device, require an app protection policy, block legacy auth).

> ***Scope boundary — read this.*** *This solution **wires the app to route to Entra ID / Conditional Access**; it does **not** build the Conditional Access policies themselves. The full CA build (break-glass accounts, report-only piloting, baseline policies) is delivered by the dedicated **Conditional Access Integration** solution. Here you create the app config that sends sign-in to Entra, then reference an existing CA policy (or the CA solution) to enforce it.*

The worked example — SAP Ariba Procurement

The guide implements one end-to-end example: the **SAP Ariba Procurement** mobile app. The goal — a user installs Ariba, opens it, and lands **pre-pointed at your Ariba site/realm/environment with the correct role option, on the Entra ID sign-in screen**, with no manual entry. The Implementation section builds this policy click-by-click in Intune for **iOS/iPadOS** (as a plist) and shows the Android managed-configuration equivalent, with every object named (e.g. `APPCFG - Ariba - iOS - Pilot`). Because the exact Ariba configuration **key names** are vendor-defined and must be taken from SAP's documentation, the guide presents them in a clearly-labeled table and flags them **"verify against SAP Ariba documentation"** rather than asserting invented keys.

How app config interacts with App Protection and supersedence

- **App config + App Protection are independent and complementary.** One configures the app; the other protects its data. A managed app can carry **both** at once — e.g. Ariba/Outlook receives its settings from an App Configuration Policy and its copy/paste/PIN controls from an App Protection Policy. They are assigned separately and do not conflict.
- **Supersedence is an *app-deployment* concept, not an app-config one.** When you replace one app package with a newer/different one via app supersedence, the **assignments carry to the superseding app but app configuration does not automatically follow** — confirm the App Configuration Policy still targets the current app after any supersedence, or the new app installs unconfigured.

Benefits

- **Zero-touch first launch** — apps configure themselves; no user data entry, fewer help-desk tickets.
- **Consistent, auditable settings** — every device/user gets the same hostname, realm, and routing.
- **Sign-in routed to your IdP** — apps land on Entra ID, so Conditional Access can enforce access.
- **Cross-platform, one method** — the same design pattern for iOS/iPadOS and Android Enterprise.
- **Repeatable** — assign the policy to a group; new users/devices inherit configuration automatically.

Limitations (be honest with buyers)

- **The app must support managed configuration.** App config only works for apps that declare supported keys (iOS Managed App Configuration, Android managed configurations, or the Intune App SDK). An app with no supported keys cannot be configured this way.
- **Vendor keys are authoritative and can drift.** Key names come from the app vendor; a wrong or outdated key is silently ignored. Always verify against current vendor documentation.
- **Channel must match posture.** A managed-device policy needs an **enrolled** device; a managed-app policy needs a **MAM/App-SDK** app. Sending the wrong channel means the config never applies.
- **Not every app is MDM-certified on every platform.** Some vendors support iOS managed configuration

but not Android (SAP states the Ariba Procurement app is not certified with MDM on Android as of current documentation — **verify against SAP Ariba documentation**). Confirm platform support before designing.

• **SSO routing depends on the app.** Whether an app can be *forced* to federated/SSO sign-in via a key depends on the app's own capabilities; some apps only route to your IdP once the user enters a work email/domain.

• **Conditional Access enforcement requires Entra ID P1** and the CA policy itself (this solution routes to it; the CA solution builds it).

The standard deployment model

> *Choose the channel (managed devices vs managed apps) → author the key/value pairs (iOS plist / Android schema) → create the App Configuration Policy and assign to a **pilot** group → validate on a device that the app opens pre-configured and routes to Entra ID / Conditional Access → expand in waves. Pattern: **Assess** → **Configure** → **Pilot** → **Validate** → **Expand.***

2. Requirements

Each requirement below lists **Description · Why required · How to verify · How to implement if missing.** Do not assume any prerequisite already exists — check each one. (Requirements common to every Intune solution — tenant, licensing, admin roles, MDM authority — are in `global-requirements-intune-solutions.md`; this file covers what is specific to App Configuration Policies.)

2.1 Microsoft Intune plan and Entra ID licensing

• **Description:** Intune (standalone or via Microsoft 365 E3/E5, EMS E3/E5, or Business Premium) for every targeted user/device, and **Microsoft Entra ID P1** where you will enforce Conditional Access on the app's sign-in.

• **Why required:** App Configuration Policies are an Intune feature. The SSO-routing outcome is only *enforced* when a Conditional Access policy (Entra ID P1) is scoped to the app.

• **How to verify:** In the Microsoft 365 admin center → **Billing** → **Licenses**, confirm Intune and Entra ID P1 are assigned. In the Intune admin center → **Apps** → **Monitor**, confirm you can open **App configuration policies**.

• **How to implement if missing:** Assign the Intune and Entra ID P1 licenses to the pilot users/devices.

2.2 Enrollment posture matched to the delivery channel

• **Description:** Decide, per app, whether you will use the **managed-device** channel (enrolled devices) or the **managed-app** channel (MAM). Have the matching posture in place.

• **Why required:** A managed-device App Configuration Policy applies **only** to enrolled devices; a

managed-app policy applies **only** to apps that integrate the Intune App SDK. Mismatched posture = the policy silently never applies.

• **How to verify:**

• Managed-device iOS/iPadOS: **Devices** → **iOS/iPadOS** shows the target device as **enrolled** and **managed**.

• Managed-device Android: **Devices** → **Android** shows the device under an **Android Enterprise** enrollment (work profile / fully managed / dedicated), and managed Google Play is connected.

• Managed-app (MAM): the app is one of the **Intune App SDK-enabled** apps and the user is licensed; enrollment is optional.

• **How to implement if missing:** Enroll the pilot device (see the relevant enrollment solution), or confirm the app supports the App SDK (managed-app) channel.

2.3 Apple MDM push certificate (iOS/iPadOS managed-device channel)

• **Description:** A current **Apple Push Notification service (APNs)** certificate connecting Intune to Apple for iOS/iPadOS management.

• **Why required:** iOS Managed App Configuration (the plist payload) is delivered over the Apple MDM channel, which requires a valid APNs certificate.

• **How to verify: Devices** → **Enrollment** → **Apple** → **Apple MDM Push certificate** shows **Active** and a future expiry.

• **How to implement if missing:** Create/renew the APNs certificate (Devices → Enrollment → Apple → Apple MDM Push certificate → follow the download-CSR / Apple Push Portal steps). **Always renew the *same* certificate** (using the same Apple ID) — a new certificate re-enrolls every device.

2.4 Managed Google Play connection (Android Enterprise managed-device channel)

• **Description:** Intune bound to **managed Google Play** so Android Enterprise apps and their **managed-configuration schemas** are available.

• **Why required:** Android managed configurations are read from the app's schema in managed Google Play; without the binding, the app and its schema do not appear in Intune.

• **How to verify: Devices** → **Enrollment** → **Android** → **Managed Google Play** shows the account as **Connected**, and the target app appears under **Apps** → **Android**.

• **How to implement if missing:** Connect managed Google Play (Devices → Enrollment → Android → Managed Google Play → follow the bind flow with a corporate Google account, not a personal Gmail).

2.5 The target app added to Intune, identified by exact publisher + bundle/package ID

• **Description:** The app you will configure is already added to Intune as a managed app, selected by its **exact store publisher** and **bundle ID (iOS)** / **package ID (Android)**.

• **Why required:** An App Configuration Policy is bound to a specific app object. Store search returns near-identical results; the exact publisher + ID guarantees you configure the right app. For the worked example, **SAP Ariba Procurement**:

• **iOS/iPadOS:** publisher **SAP SE**, bundle ID `com.sap.ariba.procurement` — *verify the exact bundle ID in the App Store listing / SAP documentation before use* (App Store item id `1451570638`).

• **Android:** SAP states the Ariba Procurement app is **not certified with MDM on Android** as of current documentation — **verify against SAP Ariba documentation**; the Android managed-config worked example is therefore shown with clearly-labeled placeholders.

• **How to verify:** **Apps** → **iOS/iPadOS** (or **Android**) lists the app with the expected publisher and bundle/package ID.

• **How to implement if missing:** Add the app: **Apps** → **iOS/iPadOS** → **Add** → **iOS store app** (or **Android** → **Add** → **Managed Google Play app**), search by the exact publisher, confirm the bundle/package ID, and save.

2.6 The vendor's supported configuration keys

• **Description:** The **exact key names and value formats** the app supports for managed configuration, taken from the vendor's documentation (and, on Android, from the managed-config schema surfaced in Intune).

• **Why required:** Keys are vendor-defined. A key the app did not publish is ignored; a typo is ignored. You cannot author a correct policy without the authoritative key list.

• **How to verify:** The vendor's admin/MDM documentation lists the keys; on Android, the Intune **configuration designer** displays the schema's keys automatically. For **SAP Ariba**, the authoritative source is the *SAP Ariba Procurement Mobile App Guide* → **"Configuring SAP Ariba Procurement mobile app in Mobile Device Management Tool."** Treat the Ariba keys in this guide as **placeholders to confirm there** — flagged **"verify against SAP Ariba documentation."**

• **How to implement if missing:** Obtain the vendor's MDM/app-config documentation before building the policy; do not proceed with guessed keys.

2.7 Microsoft Entra single sign-on plumbing (for SSO routing)

• **Description:** The identity plumbing that lets the app route sign-in to Entra ID: **MSAL**-based

auth in the app, plus the **Microsoft Enterprise SSO plug-in** on iOS (via Microsoft Authenticator / Company Portal) or the Android broker, and — where the app federates — the app registered/enabled for SSO in Entra.

• **Why required:** App configuration can *point* the app at your tenant, but the sign-in only lands on Entra ID (and inherits Conditional Access) when the SSO plumbing is present.

• **How to verify: Devices** → **Configuration** shows an **iOS/iPadOS SSO app extension** (Microsoft Enterprise SSO) profile assigned; Microsoft Authenticator/Company Portal is deployed to the device; in **Entra** → **Enterprise applications**, the target app (e.g. Ariba) exists for SSO if it federates.

• **How to implement if missing:** Deploy Microsoft Authenticator + the iOS Enterprise SSO extension profile (Devices → Configuration → Create → iOS/iPadOS → **SSO app extension**, type **Microsoft Entra ID**), and configure the app's SSO in Entra per the vendor's federation guide.

2.8 A Conditional Access policy scoped to the app (enforcement — deferred)

• **Description:** A Conditional Access policy targeting the app's cloud/enterprise application, so the routed sign-in is actually governed (require MFA / compliant device / app protection).

• **Why required:** App configuration routes sign-in to Entra; **Conditional Access is what enforces** the access decision.

• **How to verify: Entra** → **Protection** → **Conditional Access** shows a policy targeting the app (e.g. Ariba) — even if in **Report-only** while piloting.

• **How to implement if missing:** Build it with the dedicated **Conditional Access Integration** solution (break-glass accounts, report-only piloting, baseline controls). This solution intentionally **does not** build the CA policy; it wires the app to route to it.

2.9 A pilot Entra security group

• **Description:** An Entra security group holding the pilot users (managed-app channel) or pilot devices (managed-device channel), named e.g. `APPCFG - Ariba - Pilot`.

• **Why required:** Every App Configuration Policy is assigned to a group; piloting first is how you prove the config before production.

• **How to verify: Entra** → **Groups** shows the group with the intended pilot members.

• **How to implement if missing:** Create it: **Entra** → **Groups** → **New group** → **Security**, name it `APPCFG - Ariba - Pilot`, add the pilot members.

2.10 An admin workstation with Microsoft Graph PowerShell (for the included scripts)

• **Description:** A workstation with the **Microsoft.Graph** PowerShell SDK to run the five included

tools, and the delegated scopes they request.

• **Why required:** The scripts pre-flight scopes, export policies, and validate delivery via Microsoft Graph.

• **How to verify:** `Get-Module -ListAvailable Microsoft.Graph.Authentication` returns a version; the signed-in admin can consent to `DeviceManagementApps.Read.All` and `DeviceManagementConfiguration.Read.All`.

• **How to implement if missing:** `Install-Module Microsoft.Graph -Scope CurrentUser`. For **US Government** tenants, connect with `Connect-MgGraph -Environment USGov` (see §2.11).

2.11 US Government (GCC / GCC High / DoD) endpoints — if applicable

• **Description:** If your tenant is a US sovereign cloud, use the government portals and Graph environment, not the commercial ones.

• **Why required:** Government tenants use isolated endpoints. The App Configuration Policies **feature is the same**, but the admin surfaces differ.

• **How to verify / implement:**

• Admin center: `https://intune.microsoft.us` (not `intune.microsoft.com`).

• Entra: `https://entra.microsoft.us` / sign-in at `login.microsoftonline.us`.

• Graph scripts: `Connect-MgGraph -Environment USGov` (targets `graph.microsoft.us`); device service endpoints use `manage.microsoft.us`.

• The build steps, keys, plist format, and Android schema behavior are otherwise identical. No separate Government deliverable is required for this solution; any GCC High parity question that arises for a specific app should be flagged **"verify for GCC High."**

Global Requirements — Intune Solutions (category-level)

These requirements apply to **every** solution in the **Intune Solutions** category. They are published once at the category level so individual listings stay small. Confirm these before deploying *any* Intune solution. Each entry: **Description · Why · Verify · Implement-if-missing.**

G1. Microsoft Entra ID tenant

Description: An active Microsoft Entra ID (Azure AD) tenant. **Why:** It is the identity backbone for every cloud and Intune-managed resource. **Verify:** Sign in to `https://entra.microsoft.com` as an administrator. **Implement:** Create a Microsoft 365/Azure subscription, which provisions a tenant.

G2. Microsoft Intune subscription

Description: An Intune-bearing license plan. **Why:** Intune delivers all configuration, compliance, and security policies. **Verify:** `https://intune.microsoft.com` loads; **Tenant administration** → **Tenant status** shows active. **Implement:** Assign Microsoft 365 E3/E5, Business Premium, or standalone Intune Plan 1.

G3. Administrative roles

Description: Appropriate Entra/Intune admin roles for the operator. **Why:** Creating policies and enrollment settings requires elevated rights. **Verify:** Entra → **Roles and administrators**; confirm **Intune Administrator**. **Implement:** Assign **Intune Administrator**; reserve Global Administrator for tenant-wide changes.

G4. Device enrollment in Intune

Description: Targeted Windows devices are enrolled and manageable. **Why:** Policies only reach enrolled devices. **Verify:** Intune → **Devices** → **All devices**; the device appears and is **compliant**. **Implement:** Enable **Automatic Enrollment** and join devices to Entra ID.

G5. Pilot security group

Description: A Microsoft Entra security group for pilot/targeted users or devices. **Why:** Every solution is piloted before fleet rollout. **Verify:** Entra → **Groups**; the pilot group exists. **Implement:** Entra → **Groups** → **New group** → **Security** → add members.

G6. Supported Windows baseline

Description: Windows 10 21H2+ or Windows 11 with current updates. **Why:** Modern Intune features assume a patched, supported OS. **Verify:** `winver`, or Intune → device → **Hardware**. **Implement:** Deploy **Update rings** or upgrade to Windows 11.

G7. Network access to Microsoft cloud endpoints

Description: Outbound HTTPS to Microsoft 365, Entra, and Intune service URLs. **Why:** Enrollment, policy sync, and authentication depend on these endpoints. **Verify:** From a device, reach `https://login.microsoftonline.com` and `https://enrollment.manage.microsoft.com`. **Implement:** Allow Microsoft's published Office 365/Intune URL & IP ranges through proxy/firewall.

The standard deployment model (all solutions): Assess → Configure → Pilot → Validate → Expand.

3. Implementation

This section builds App Configuration Policies **click-by-click** for **iOS/iPadOS** and **Android Enterprise**, using **SAP Ariba Procurement** as the worked example, and wires the app's sign-in to route to Microsoft Entra ID / Conditional Access. Every object you create is named; reuse the exact names in Testing and Troubleshooting.

> ***Portal note (US Government):*** *all navigation below uses the commercial admin center* `https://intune.microsoft.com`*. On GCC/GCC High use* `https://intune.microsoft.us` *— the blades and steps are identical.*
>
> ***Vendor-key note (read before you build the Ariba example):*** *the Ariba configuration* ***key names*** *shown here are* ***placeholders****. SAP defines the authoritative keys in the *SAP Ariba Procurement Mobile App Guide** → ***"Configuring SAP Ariba Procurement mobile app in Mobile Device Management Tool."*** *Confirm every key there before production use — flagged* ***"verify against SAP Ariba documentation."*** *The *mechanics* below (how to build, assign, and validate a policy) are correct regardless of the exact keys.*

Phase A — Assess and choose the channel

Step 1 — Decide managed-device vs managed-app, per app

For each app you will configure, choose one channel (see Overview §"The two delivery channels"):

- **Managed devices (MDM):** device is **enrolled**; you want the app configured on the device.

iOS = plist payload; Android = managed configurations.

- **Managed apps (MAM):** device may be **unenrolled** (BYOD); the app integrates the Intune App SDK;

configuration is tied to the **user**.

For the Ariba worked example we use the **managed-device (MDM)** channel on **iOS/iPadOS** (SAP supports generic MDM on iOS; Ariba is **not MDM-certified on Android** per current SAP docs — **verify against SAP Ariba documentation**). The Android steps (Phase D) show the general managed-configuration method with a placeholder Ariba schema so the pattern is complete.

Step 2 — Confirm the app object and gather the keys

1. In **Apps** → **iOS/iPadOS**, confirm **SAP Ariba Procurement** is present — publisher **SAP SE**,

bundle ID `com.sap.ariba.procurement` (*verify the exact bundle ID in the App Store listing / SAP docs*). If missing, add it (Requirements §2.5).

1. Open the *SAP Ariba Procurement Mobile App Guide* MDM section and record the **exact key names**

for: site/server **hostname**, **realm**, **environment**, **user role**, and any **SSO/sign-in** key. Enter them in the key table (Step 5). **verify against SAP Ariba documentation.**

Step 3 — Create the pilot group

1. **Entra admin center** → **Groups** → **New group.**

2. Group type **Security**; **Name** = `APPCFG - Ariba - Pilot`; add the pilot **devices** (managed-

device channel) or pilot **users** (managed-app channel).

1. **Create.**

Phase B — Build the iOS/iPadOS App Configuration Policy (Ariba, managed devices)

Step 4 — Create the policy and bind it to the app

1. **Intune admin center (`https://intune.microsoft.com`) → Apps → App configuration policies →

Create → Managed devices.**

1. **Basics:**

• **Name:** `APPCFG - Ariba - iOS - Pilot`

• **Description:** `Pre-configures SAP Ariba Procurement (site/realm/environment/role) and routes sign-in to Entra ID. Pilot.`

• **Platform: iOS/iPadOS**

• **Targeted app:** click **Select app**, choose **SAP Ariba Procurement** (publisher **SAP SE**,

bundle ID `com.sap.ariba.procurement`). Click **Select**, then **Next**.

Step 5 — Author the settings (Configuration designer — preferred)

1. On the **Settings** tab, set **Configuration settings format = Use configuration designer**.

2. Add one row per vendor key. **Use hostnames, not URLs, in any "server/host" field** — no `https://`

and no trailing path/slash (a path is sent to DNS as one string and fails to resolve).

Configuration key (PLACEHOLDER — verify against SAP Ariba documentation)	Value type	Example value	Notes
`aribaSiteHost`	String	`s1.ariba.com`	**Hostname only** — no `https://`, no path. Region-specific (e.g. `s1-eu.ariba.com`); confirm your site host.
`aribaRealm`	String	`yourcompany`	The buyer **realm/subdomain** (the `realm=` value from your Ariba SSO/ACS URL).
`aribaEnvironment`	String	`production`	e.g. `production` / `test` — confirm the exact accepted values.
`userRole`	String	`requester`	The employee/role option pre-selected on launch — confirm accepted values.

Configuration key (PLACEHOLDER — verify against SAP Ariba documentation)	Value type	Example value	Notes
`authMode`	String	SSO	Route sign-in to the federated/Entra ID flow rather than local sign-in — confirm the exact key/value.
`tenantId`	String	`{{tenantid}}`	Optional — Entra tenant GUID; the `{{tenantid}}` token is expanded by Intune at delivery.

> *Intune supports **token variables** in values — e.g. `{{userprincipalname}}`, `{{mail}}`, `{{tenantid}}`, `{{deviceid}}` — expanded per user/device at delivery. Use `{{mail}}` or `{{userprincipalname}}` to pre-fill a username field so the user only enters credentials.*

1. Click **Next**.

Step 5-ALT — Author the settings (Enter XML data — plist)

If SAP's documentation supplies the configuration as a property list, choose **Configuration settings format = Enter XML data** and paste a **valid Apple plist**. It **must** be a lowercase `<dict>` root of `<key>/<string>` pairs with **no** `xmlns/xsi` attributes and **no** vendor XML profile header (a capitalized `<Dict>` or a namespaced document is rejected):

```
<dict>
    <key>aribaSiteHost</key>
    <string>s1.ariba.com</string>
    <key>aribaRealm</key>
    <string>yourcompany</string>
    <key>aribaEnvironment</key>
    <string>production</string>
    <key>userRole</key>
    <string>requester</string>
    <key>authMode</key>
    <string>SSO</string>
    <key>tenantId</key>
    <string>{{tenantid}}</string>
</dict>
```

> *The `<?xml … ?>` line and the `<!DOCTYPE plist …>/<plist>` wrapper are added by the platform — if you paste them and the portal complains ("Remove XML headers from property list"), delete them and paste only from the `<dict>` to the closing `</dict>`. Keys/values remain **placeholders — verify against SAP Ariba documentation.** A full vendor *profile* document, if the app supports one, goes in as a **string value under a key** (CDATA), never as the plist's structure.*

Step 6 — Assign to the pilot group

1. On the **Assignments** tab → **Included groups** → **Add groups** → select **`APPCFG - Ariba - Pilot` → Select**.

1. **Next** → **Review + create** → **Create.** The policy now applies to enrolled iOS devices in the pilot

group the next time they sync and the app is present.

Phase C — Wire sign-in to Entra ID / Conditional Access

Step 7 — Ensure the Entra SSO plumbing is on the device

1. **Deploy Microsoft Authenticator** (Apps → iOS/iPadOS → Add → iOS store app → **Microsoft Authenticator**, **publisher** Microsoft Corporation**, bundle ID** `com.azure.authenticator`**), assigned to the pilot group as** Required**.

1. **Create the iOS Enterprise SSO extension profile** (if not already present):

• **Devices** → **Configuration** → **Create** → **New policy** → **iOS/iPadOS** → **Templates** → **Device features.**

• **Name:** `APPCFG - iOS - Enterprise SSO`.

• Under **Single sign-on app extension**, set **SSO app extension type = Microsoft Entra ID**.

• **Save**, then **Assign** to `APPCFG - Ariba - Pilot`. This delivers the Microsoft Enterprise SSO plug-in so the app's MSAL sign-in lands on Entra ID silently.

Step 8 — Confirm the app routes to Entra (config side)

The `authMode`/SSO key from Step 5 tells Ariba to use federated sign-in. Combined with the Enterprise SSO extension and the pre-filled `aribaRealm`, the app opens straight to the **Entra ID** sign-in for your tenant. (Whether an app can be *forced* to SSO via a key is app-dependent — **verify against SAP Ariba documentation**; some apps route to the IdP only after the user's work email/realm is detected, which the pre-filled realm satisfies.)

Step 9 — Reference the Conditional Access policy (do not build it here)

1. **Entra admin center** → **Protection** → **Conditional Access** → **Policies.** Confirm a policy targets the Ariba enterprise application (or the appropriate cloud app) with your controls (require MFA / require compliant device / require app protection policy).

1. If none exists, **build it with the dedicated Conditional Access Integration solution** — create break-glass accounts, pilot in **Report-only**, then enable. This solution intentionally stops at *routing* sign-in to Entra; the CA solution *enforces* it.

> ***Boundary reminder:*** *naming here — reference a CA policy such as* `CA - Ariba - Require Compliant + MFA` *if your CA solution created one, but do* ***not*** *author CA logic in this guide.*

Phase D — Build the Android Enterprise managed configuration (general pattern)

> *SAP states the Ariba Procurement app is* ***not certified with MDM on Android*** *as of current docs (****verify against SAP Ariba documentation****). The steps below are the* ***general*** *Android managed-configuration method — correct for any Android Enterprise app that publishes a managed-configuration schema (Outlook, Chrome, and most enterprise apps). The Ariba keys are*

*shown as **placeholders** to illustrate the pattern.*

Step 10 — Create the Android managed-configuration policy

1. **Apps → App configuration policies → Create → Managed devices.**
2. **Basics:**

• **Name:** `APPCFG - Ariba - Android - Pilot`

• **Platform: Android Enterprise**

• **Profile type: All Profile Types** (or the specific work-profile/fully-managed type you target).

• **Targeted app: Select app** → choose the managed Google Play app (for a real deployment, the vendor's Android app — publisher and package ID confirmed). **Next.**

Step 11 — Set the keys from the app's schema

1. On **Settings**, set **Configuration settings format = Use configuration designer**. Intune reads the app's **managed-configuration schema** from managed Google Play and lists its keys — **you do not type key names on Android; you fill in the schema's values.**
1. Set the values (placeholders shown; the real key names come from the app's schema):

Schema key (from managed Google Play — example/placeholder)	Value	Notes
`server_host`	`s1.ariba.com`	**Hostname only.**
`realm`	`yourcompany`	Buyer realm/subdomain.
`environment`	`production`	Confirm accepted values.
`user_role`	`requester`	Confirm accepted values.
`auth_mode`	`sso`	Route to federated/Entra ID sign-in.

*If the schema exposes a key as an enum/bool/bundle, the designer shows the correct control — use it rather than free text. The **JSON** channel (below) is only for the rare case you must supply a raw managed-configuration document.*

1. **Next.**

Step 11-ALT — Android via raw JSON (only if required)

If you must supply the configuration as raw JSON (e.g. an OEMConfig-style bundle), the values map to the schema's keys exactly:

```
{
  "kind": "androidenterprise#managedConfiguration",
  "managedProperty": [
    { "key": "server_host", "valueString": "s1.ariba.com" },
    { "key": "realm", "valueString": "yourcompany" },
    { "key": "environment", "valueString": "production" },
    { "key": "user_role", "valueString": "requester" },
    { "key": "auth_mode", "valueString": "sso" }
  ]
```

}

Keys remain **placeholders — verify against the app's managed-configuration schema.**

Step 12 — Assign the Android policy

1. **Assignments** → **Included groups** → **Add groups** → `APPCFG - Ariba - Pilot` → **Select**.
2. **Review + create** → **Create.**

Phase E — Managed-app (MAM) channel — when the device is unenrolled

If you need to configure a MAM app (Intune App SDK) on an **unenrolled/BYOD** device (e.g. Outlook on a personal phone), use the **managed apps** channel instead of managed devices:

Step 13 — Create a managed-app configuration policy

1. **Apps** → **App configuration policies** → **Create** → **Managed apps.**
2. **Basics: Name** = `APPCFG - Outlook - MAM - Pilot`. Under **Public apps** → **Select public apps**,

choose the App SDK-enabled app(s) (e.g. **Microsoft Outlook**). **Next.**

1. **Settings:** add the app's supported managed-app keys (from vendor docs). **Next.**
2. **Assignments:** assign to a **user** group (MAM policies target users, not devices) —

`APPCFG - Ariba - Pilot` if it holds users. **Create.**

> *Managed-app policies apply on enrolled **and** unenrolled devices because they ride the App SDK, not the OS MDM payload. This is the channel to use when there is no enrollment to carry an OS plist.*

Phase F — Pilot → Validate → Expand

Step 14 — Validate on a pilot device

Run the Testing procedures (§5): on a pilot device, install/open the app and confirm it is **pre-configured** (host/realm/environment/role filled) and lands on the **Entra ID / Conditional Access** sign-in. Use `APPCFG-Validate-Assignment.ps1` to confirm the device/user received the policy.

Step 15 — Expand to production

1. Create production groups (e.g. `APPCFG - Ariba - Prod` for devices/users) and a production policy

copy named `APPCFG - Ariba - iOS - Prod` (and …`- Android - Prod`).

1. Add the production group to the production policy's **Assignments**; expand in waves.
2. Keep the pilot policy for future changes; test key changes in pilot before promoting.

Step 16 — Document and hand off

Record every object name (`APPCFG - Ariba - iOS - Pilot`, `APPCFG - Ariba - Android - Pilot`, `APPCFG - iOS - Enterprise SSO`, `APPCFG - Ariba - Pilot` group, production copies) and the confirmed vendor keys, so the configuration is auditable and reproducible.

5. Testing & Validation

Validate on a **pilot device** before expanding. Each test states the action and an explicit **PASS/FAIL** gate. Reuse the object names from Implementation (`APPCFG - Ariba - iOS - Pilot`, `APPCFG - Ariba - Android - Pilot`, `APPCFG - Ariba - Pilot` group, `APPCFG - iOS - Enterprise SSO`).

The end-to-end acceptance test (the one that matters)

Scenario: a pilot user picks up an enrolled device, opens SAP Ariba Procurement for the first time, and lands **pre-configured** on the **Entra ID / Conditional Access** sign-in — with no manual data entry.

Test 1 — Policy targets the device/user

- **Action (iOS):** Intune → **Apps → App configuration policies → `APPCFG - Ariba - iOS - Pilot` →

Device install status **(or** Assignments**), or run `APPCFG-Validate-Assignment.ps1` for the pilot device.

- **PASS:** the pilot device appears as targeted/received.
- **FAIL:** device not listed → device not in `APPCFG - Ariba - Pilot`, or not enrolled → fix

membership/enrollment and re-sync.

Test 2 — App opens pre-configured (the zero-touch gate)

- **Action:** On the pilot device, install (or reinstall) and **open SAP Ariba Procurement**.
- **PASS:** the app opens with the **site host / realm / environment / role** already populated — the

user is **not** asked to type a server or realm.

- **FAIL:** the app asks for server/realm → config not applied (see Test 4) or a key name is wrong

(see Troubleshooting "config not applying / key typos").

Test 3 — Sign-in routes to Entra ID and Conditional Access applies

- **Action:** Proceed to sign in from the pre-configured app.
- **PASS:** the sign-in screen is your **Entra ID** tenant; Conditional Access controls fire as

expected (e.g. MFA prompt, or compliant-device requirement); access is granted only when controls are satisfied.

- **FAIL — lands on a non-Entra/local login:** the `authMode`/SSO key or Enterprise SSO extension is

missing/misconfigured (Troubleshooting "not routing to Entra ID").

- **FAIL — blocked unexpectedly:** the Conditional Access policy (owned by the CA solution) is

mis-scoped — review it there, ideally in **Report-only** first.

Test 4 — Confirm the delivered values on the device

• **Action (iOS):** on the device, **Settings → General → VPN & Device Management → (your MDM profile) **shows the app configuration is present; or in Intune the policy's status shows** Succeeded** for the device. Optionally check the app's in-app "About/Diagnostics" screen for the active host/realm.

• **PASS:** the values match what you authored (hostname has **no** `https://` or path).

• **FAIL:** a value shows a URL/path in a host field → correct it to a **hostname only** and re-test.

Test 5 — Android managed configuration (if used)

• **Action:** On an Android Enterprise pilot device, open the target app; in Intune check `APPCFG - Ariba - Android - Pilot` → **status**.

• **PASS:** status **Succeeded**; the app reads the schema values.

• **FAIL — keys greyed/absent:** the app did not publish that schema key → only set keys the schema exposes (Android keys are schema-authoritative).

Test 6 — Managed-app (MAM) channel (if used)

• **Action:** On an unenrolled/BYOD device, sign in to the MAM app (e.g. Outlook) with the work account; confirm the managed-app configuration applied.

• **PASS:** the app reflects the pushed settings for the work identity.

• **FAIL:** verify the policy is **Managed apps** (not Managed devices) and assigned to a **user** group, and the app is App SDK-enabled.

Test 7 — App Protection coexistence (if both are assigned)

• **Action:** With an App Protection Policy also assigned, open the app.

• **PASS:** the app is both **configured** (host/realm/role) **and** protected (PIN/copy-paste controls) — the two policies coexist without conflict.

• **FAIL:** if protection blocks launch, that is an App Protection issue, not app config — review the APP conditional-launch settings in the App Protection solution.

Test 8 — Reporting coverage

• **Action:** Run `APPCFG-Report-AppConfig.ps1`.

• **PASS:** every intended app shows an attached App Configuration Policy; apps that should be configured but are not are listed for follow-up.

• **FAIL:** an intended app shows no config → create/assign its policy.

Exit criteria to expand from pilot to production

• Tests 1–4 PASS on at least two pilot devices (and Test 5/6 for the channels you use).

- The app opens pre-configured with **no** manual server/realm entry (Test 2).
- Sign-in lands on **Entra ID** and Conditional Access behaves as designed (Test 3).
- Reporting (Test 8) shows the intended coverage with no unexpected gaps.

6. Troubleshooting

Symptom → cause → fix. Reuse the object names from Implementation.

Portal / plist errors

"Property list must begin and end with 'dict' tag"

- **Cause:** the pasted XML root is a capitalized `<Dict>`, or is wrapped in something other than a lowercase `<dict>` (e.g. a vendor `<Configuration>`/`<profile>` root).
- **Fix:** the iOS "Enter XML data" content is an **Apple plist**. Make the root a lowercase `<dict>` … `</dict>` containing only `<key>`/`<string>` (etc.) pairs. Paste only from `<dict>` to `</dict>`.

"Remove XML headers from property list"

- **Cause:** you pasted the `<?xml …?>` declaration and/or a `<!DOCTYPE plist …>` / `<plist>` wrapper, or the payload carries `xmlns`/`xsi` **namespace attributes** (a vendor XML *profile* schema).
- **Fix:** delete the XML/DOCTYPE/plist headers and all namespace attributes — paste **only** the `<dict>…</dict>`. Managed App Configuration is **not** a transport for a vendor's namespaced XML profile. If the app truly needs a whole profile document, put it in as a **string value under a key** (CDATA), never as the plist structure.

The portal rejects the plist but the XML "looks fine"

- **Cause:** a mismatched tag, a `<key>` with no following value element, or a smart-quote pasted from a Word doc.
- **Fix:** validate the plist (every `<key>` has exactly one value element; tags balanced; straight ASCII quotes). Prefer the **Configuration designer** to avoid hand-editing XML entirely.

Config not applying

App opens and still asks for server/realm (nothing pre-filled)

- **Causes & fixes (work top to bottom):**

1. **Wrong channel.** A **managed-device** policy only applies to **enrolled** devices; on an unenrolled/BYOD phone it never applies. Confirm the device is enrolled, or use the **managed apps (MAM)** channel instead. (See "MAM vs MDM confusion" below.)
1. **Device/user not in the assigned group.** Run

`APPCFG-Validate-Assignment.ps1 -PolicyName "APPCFG - Ariba - iOS - Pilot" -DeviceName <name>`. If FAIL, add the device/user to `APPCFG - Ariba - Pilot` and re-sync.

1. **Key name typo / wrong key.** A key the app did not publish is silently ignored, so the field stays empty. Re-check every key against the vendor's documentation (Ariba: the SAP MDM guide — **verify against SAP Ariba documentation**). On Android, only keys the **schema** exposes work.

1. **Policy not synced yet.** Force a sync (Company Portal → **Sync**, or Intune → device → **Sync**) and reopen the app; some apps read config only on a fresh launch or reinstall.

1. **App doesn't support managed configuration at all.** Confirm the app declares iOS Managed App Configuration / Android managed configurations / Intune App SDK. If not, app config cannot configure it.

Some keys apply, others don't

• **Cause:** the ones that don't are misnamed, use the wrong value **type** (string vs integer vs bool), or aren't in the app's schema.

• **Fix:** match each key's name and type exactly to the vendor schema. On Android use the designer's control (enum/bool/bundle) rather than free text.

The app went to the wrong environment/site

• **Cause:** a **URL in a hostname field** (e.g. `https://s1.ariba.com/Buyer`) or a region-wrong host.

• **Fix:** set the host to a **bare hostname** — no `https://`, no path (a path is sent to DNS as one string → NXDOMAIN / wrong routing). Confirm the region-specific host (e.g. `s1-eu.ariba.com`).

MAM vs MDM confusion

"I created an App Configuration Policy but it doesn't reach my BYOD users"

• **Cause:** you created a **Managed devices** policy, but the target phones are **unenrolled** — the OS payload channel doesn't exist there.

• **Fix:** create a **Managed apps** policy instead (Apps → App configuration policies → Create → **Managed apps**), scope it to the **user** group, and confirm the app is Intune App SDK-enabled (e.g. Outlook). Managed-app policies apply on enrolled *and* unenrolled devices.

"My managed-app policy isn't applying to a device group"

• **Cause:** managed-app (MAM) policies target **users**, not devices.

• **Fix:** assign the policy to a **user** group; device-group assignment does not drive MAM config.

SSO / Conditional Access routing

App lands on a local/non-Entra login instead of Entra ID

• **Causes & fixes:**

1. The `authMode`**/SSO** key is missing or set to a local value — set it to route to federated/SSO

sign-in (**verify against SAP Ariba documentation** for the exact key/value).

1. The **Microsoft Enterprise SSO extension** isn't on the device — deploy the `APPCFG - iOS - Enterprise SSO` profile and Microsoft Authenticator to the pilot group.

1. The **realm/tenant isn't pre-filled**, so the app can't detect your IdP — confirm `aribaRealm` (or the vendor's realm/tenant key) is delivered.

Sign-in reaches Entra but the user is blocked (or not challenged as expected)

- **Cause:** this is **Conditional Access**, which this solution routes to but does **not** build.
- **Fix:** review the CA policy targeting the app in the dedicated **Conditional Access Integration** solution — pilot it in **Report-only**, confirm break-glass exclusions, then enforce. Do not troubleshoot CA logic here.

Supersedence & coexistence

After replacing/superseding the app, it installs unconfigured

- **Cause:** app **supersedence** carries app *assignments* but **not** App Configuration Policies to the superseding app.
- **Fix:** point the App Configuration Policy's **targeted app** at the current (superseding) app, or create a new policy bound to it.

App Protection seems to conflict with app config

- **Cause:** they don't conflict — but an App Protection **conditional launch** rule (e.g. min OS, jailbreak block) can stop the app before config is visible.
- **Fix:** confirm the app is *both* configured and protected; if the app is blocked at launch, that's an App Protection issue — adjust it in the App Protection solution, not here.

Scripts / Graph

`Connect-MgGraph` *fails or the tools see the wrong tenant/cloud*

- **Cause:** wrong cloud (Commercial vs US Government), or missing scope consent.
- **Fix:** run `APPCFG-Preflight.ps1` (add `-Environment USGov` for GCC/GCC High). It confirms the signed-in identity, tenant, cloud, and each required scope. For Government, all tools accept `-Environment USGov` (Graph `graph.microsoft.us`); the admin portal is `https://intune.microsoft.us`. Any app-specific GCC High parity question → **verify for GCC High**.

`APPCFG-Remediation.ps1` *didn't change anything*

- **Cause:** it is **safe by default** — without `-Commit` it runs in preview and writes nothing.
- **Fix:** re-run with `-Commit` once you've reviewed the previewed assignment. It preserves existing assignments (re-sends the full set), so it won't clobber other groups.

7. Best Practices

Design

- **Pick the channel first, deliberately.** Decide *managed devices* vs *managed apps* per app before

you build anything — it determines whether the policy will ever apply. Enrolled + OS payload = managed devices; unenrolled/BYOD or App-SDK apps = managed apps.

- **Never invent keys.** Key names are vendor-defined. Take iOS keys from the vendor's MDM/app-config

documentation and Android keys from the app's managed-configuration **schema** in managed Google Play. A key the app did not publish is silently ignored.

- **Prefer the configuration designer over raw XML/JSON.** The designer prevents malformed payloads.

Use "Enter XML data" (iOS plist) or raw JSON (Android) only when the vendor supplies content that way.

- **Author iOS XML as a real plist.** Lowercase `<dict>` root, `<key>`/`<string>` pairs only, **no**

`xmlns`/`xsi` attributes, **no** vendor profile header. A full vendor profile document, if supported, is a **string value under a key**, not the plist structure.

- **Every "server/host" value is a hostname, not a URL.** No `https://`, no trailing path/slash — a

path is sent to DNS as one string and fails. Region-specific hosts (e.g. `s1-eu.ariba.com`) matter; confirm the correct one.

Naming & hygiene

- **Name everything consistently:** `<Purpose> - <App> - <Platform> - <Stage>` — e.g.

`APPCFG - Ariba - iOS - Pilot`, `APPCFG - Ariba - Android - Prod`. Reuse the exact names in testing, troubleshooting, and scripts.

- **One app per policy** for clarity and clean assignment; use separate pilot/prod policies rather

than editing a live production policy in place.

- **Use Intune token variables** (`{{userprincipalname}}`, `{{mail}}`, `{{tenantid}}`) to pre-fill

per-user values instead of hardcoding — cleaner and reusable across users.

Identity & SSO

- **Deploy the Entra SSO plumbing before relying on routing:** Microsoft Authenticator + the iOS

Enterprise SSO app extension (Microsoft Entra ID type), or the Android broker. App config points the app at the tenant; the SSO extension makes the landing silent.

- **Keep Conditional Access where it belongs.** This solution routes sign-in to Entra; build and own

the CA policies in the dedicated **Conditional Access Integration** solution. Pilot CA in **Report-only** and keep **break-glass** accounts excluded.

Rollout & change control

- **Pilot first, always.** Prove the app opens pre-configured and routes to Entra on a small `Pilot` group before creating the `Prod` copy and expanding in waves.
- **Change keys in pilot, then promote.** A wrong key just doesn't apply — but a wrong *host* can send users to the wrong environment. Validate changes in pilot.
- **Mind supersedence.** App configuration does **not** automatically follow an app through supersedence. After replacing an app package, confirm the App Configuration Policy still targets the current app or the new app installs unconfigured.
- **Coexist with App Protection intentionally.** Assign App Configuration (settings) and App Protection (data controls) as separate, complementary policies; test that both apply.

Operations

- **Baseline and monitor with the scripts.** Run `APPCFG-Get-AppConfigPolicies.ps1` to export policies+assignments into source control, `APPCFG-Report-AppConfig.ps1` to spot apps missing config, and `APPCFG-Validate-Assignment.ps1` after every change.
- **Re-run pre-flight after tenant changes.** `APPCFG-Preflight.ps1` confirms scopes/roles and the correct cloud (Commercial vs USGov) before you make changes.
- **US Government:** use `https://intune.microsoft.us` and `Connect-MgGraph -Environment USGov`; the feature and steps are identical, only the endpoints change. Flag any app-specific GCC High parity question **"verify for GCC High."**

8. Deliverables Summary

What you receive

- **Implementation Solution Guide (Word + PDF)** — the complete, click-by-click package:
- **Solution Overview** — App Configuration Policies, the two delivery channels (managed devices vs managed apps), the enrolled vs MAM distinction, authoring key/value pairs (iOS plist, Android schema, designer vs enter-XML), SSO routing, and how app config coexists with App Protection and supersedence.
- **Requirements** — each prerequisite with Why / Verify / Implement-if-missing, plus the category-level global requirements and US Government endpoint notes.
- **Implementation** — build the App Configuration Policy for **iOS/iPadOS** (plist) **and Android Enterprise** (managed configurations), the **SAP Ariba Procurement** worked example (pre-fill site/realm/environment/role + route to Entra ID), the managed-app (MAM) channel, and pilot → production expansion. Every object named.

• **Testing & Validation** — pass/fail gates, centered on the end-to-end test: an enrolled device opens Ariba and lands pre-configured at the Entra ID / Conditional Access sign-in.

• **Troubleshooting** — plist errors, key typos, "config not applying," MAM-vs-MDM confusion, SSO-routing failures, supersedence, and Government-cloud gotchas.

• **Best Practices** — channel choice, vendor-key discipline, hostname-not-URL, naming, SSO plumbing, rollout, and operations.

• **Five Microsoft Graph PowerShell tools** (read-only by default, cloud-aware for Commercial and US Gov; no credentials stored):

• `APPCFG-Preflight.ps1` — confirm Graph scopes/roles and the correct cloud before changes.

• `APPCFG-Get-AppConfigPolicies.ps1` — export existing managed-device **and** managed-app App Configuration Policies with their assignments (baseline / source control).

• `APPCFG-Validate-Assignment.ps1` — confirm a specific device/user received a given policy.

• `APPCFG-Report-AppConfig.ps1` — which apps have configuration and which do not (coverage gaps).

• `APPCFG-Remediation.ps1` — guarded staging of a missing assignment; `-WhatIf` **by default**, writes only with explicit `-Commit`.

• **Templates** (generic + Ariba placeholder, clearly commented that vendor keys must be confirmed):

• `ios-appconfig.plist` — sample iOS Managed App Configuration property list (generic).

• `ios-appconfig-ariba.plist` — the Ariba placeholder plist from the worked example.

• `android-managed-config.json` — sample Android managed-configuration document (generic).

• `android-managed-config-ariba.json` — the Ariba placeholder Android managed config.

Outcome

Apps that configure themselves on first launch — pre-filled with the correct host, realm, environment, and role — and route every sign-in to Microsoft Entra ID so Conditional Access can enforce access, proven on a pilot device before you expand. Buy once, reuse across unlimited apps and devices.

Accuracy note

The SAP Ariba configuration **key names** in this package are **placeholders**. The authoritative keys live in the *SAP Ariba Procurement Mobile App Guide* → **"Configuring SAP Ariba Procurement mobile app in Mobile Device Management Tool"** — confirm them there before production use (**"verify against SAP Ariba documentation"**). The build/assign/validate **mechanics** are correct regardless of the exact keys. SAP states the Ariba Procurement app is not MDM-certified on Android as of current documentation.

Appendix A — PowerShell Scripts

Included in the accompanying ZIP. Each script's header documents full usage.

APPCFG-Get-AppConfigPolicies.ps1

Purpose: Inventories every App Configuration Policy in the tenant — both channels: managed-device policies (mobileAppConfigurations, iOS plist / Android managed configurations) and managed-app policies (targetedManagedAppConfigurations, MAM / App SDK) — with each policy's platform, targeted app(s), and group assignments. Use it to baseline your app-config posture and to keep an exported copy in source control. Read-only. HOW TO RUN powershell -ExecutionPolicy Bypass -File .\APPCFG-Get-AppConfigPolicies.ps1 -CsvPath C:\Temp\appcfg.csv powershell -ExecutionPolicy Bypass -File .\APPCFG-Get-AppConfigPolicies.ps1 -Environment USGov Requires the Microsoft Graph SDK and DeviceManagementApps.Read.All. EXPECTED RESULTS One row per policy (Channel / Name / Platform / TargetedApps / Assignments), plus optional CSV. PASS = every intended app-config policy appears with the expected platform and assignment. .NOTES Intune Solutions — read-only. Cloud-aware (Commercial / USGov). No credentials stored. .EXAMPLE .\APPCFG-Get-AppConfigPolicies.ps1 -CsvPath C:\Temp\appcfg.csv > [CmdletBinding()] param([string]$CsvPath, [ValidateSet('Global', 'USGov', 'USGovDoD')] [string]$Environment = 'Global')

How to run: powershell -ExecutionPolicy Bypass -File .\APPCFG-Get-AppConfigPolicies.ps1 -CsvPath C:\Temp\appcfg.csv powershell -ExecutionPolicy Bypass -File .\APPCFG-Get-AppConfigPolicies.ps1 -Environment USGov Requires the Microsoft Graph SDK and DeviceManagementApps.Read.All. EXPECTED RESULTS One row per policy (Channel / Name / Platform / TargetedApps / Assignments), plus optional CSV. PASS = every intended app-config policy appears with the expected platform and assignment. .NOTES Intune Solutions — read-only. Cloud-aware (Commercial / USGov). No credentials stored. .EXAMPLE .\APPCFG-Get-AppConfigPolicies.ps1 -CsvPath C:\Temp\appcfg.csv > [CmdletBinding()] param([string]$CsvPath, [ValidateSet('Global', 'USGov', 'USGovDoD')] [string]$Environment = 'Global')

APPCFG-Preflight.ps1

Purpose: Pre-flight check before creating or editing App Configuration Policies. Connects to Microsoft Graph in the requested cloud (Commercial or US Government), confirms the delegated scopes the other APPCFG tools need are granted, and prints the signed-in identity, tenant, and cloud so you know you are pointed at the right environment. Read-only; makes no changes. HOW TO RUN powershell -ExecutionPolicy Bypass -File .\APPCFG-Preflight.ps1 powershell -ExecutionPolicy Bypass -File .\APPCFG-Preflight.ps1 -Environment USGov Requires the Microsoft Graph SDK. Consents DeviceManagementApps.Read.All + DeviceManagementConfiguration.Read.All (read-only). EXPECTED RESULTS Signed-in user, tenant id, cloud, and a PASS/FAIL line per required scope. PASS = every required scope present and the expected cloud is connected. .NOTES Intune Solutions — read-only. Cloud-aware (Commercial / USGov). No credentials stored. .EXAMPLE .\APPCFG-Preflight.ps1 -Environment USGov > [CmdletBinding()] param([ValidateSet('Global', 'USGov', 'USGovDoD')] [string]$Environment = 'Global')

How to run: powershell -ExecutionPolicy Bypass -File .\APPCFG-Preflight.ps1 powershell -ExecutionPolicy Bypass -File .\APPCFG-Preflight.ps1 -Environment USGov Requires the Microsoft Graph SDK. Consents DeviceManagementApps.Read.All + DeviceManagementConfiguration.Read.All (read-only). EXPECTED RESULTS Signed-in user, tenant id, cloud, and a PASS/FAIL line per required

scope. PASS = every required scope present and the expected cloud is connected. .NOTES Intune Solutions — read-only. Cloud-aware (Commercial / USGov). No credentials stored. .EXAMPLE .\APPCFG-Preflight.ps1 -Environment USGov > [CmdletBinding()] param([ValidateSet('Global', 'USGov', 'USGovDoD')] [string]$Environment = 'Global')

APPCFG-Remediation.ps1

Purpose: Fixes the most common app-config gap: a policy that exists but is not assigned to the intended group. Resolves a managed-device App Configuration Policy by name and an Entra group by name, shows the assignment it WOULD add, and — only when you pass -Commit — adds the group to the policy's assignments (preserving existing assignments). Safe by default: with no -Commit it runs in preview (-WhatIf) and writes nothing. HOW TO RUN Preview (default, no changes): powershell -ExecutionPolicy Bypass -File .\APPCFG-Remediation.ps1 -PolicyName "APPCFG - Ariba - iOS - Pilot" -GroupName "APPCFG - Ariba - Pilot" Apply (explicit): powershell -ExecutionPolicy Bypass -File .\APPCFG-Remediation.ps1 -PolicyName "APPCFG - Ariba - iOS - Pilot" -GroupName "APPCFG - Ariba - Pilot" -Commit Requires the Microsoft Graph SDK and DeviceManagementApps.ReadWrite.All (write only used with -Commit). EXPECTED RESULTS Preview: prints the assignment that would be added. -Commit: adds it and confirms. Existing assignments are preserved (the full set is re-sent). .NOTES Intune Solutions — guarded write (-WhatIf by default; writes only with -Commit). Cloud-aware. No credentials stored. .EXAMPLE .\APPCFG-Remediation.ps1 -PolicyName "APPCFG - Ariba - iOS - Pilot" -GroupName "APPCFG - Ariba - Pilot" -WhatIf > [CmdletBinding(SupportsShouldProcess, ConfirmImpact = 'High')] param([Parameter(Mandatory)][string]$PolicyName, [Parameter(Mandatory)][string]$GroupName, [switch]$Commit, [ValidateSet('Global', 'USGov', 'USGovDoD')] [string]$Environment = 'Global')

How to run: Preview (default, no changes): powershell -ExecutionPolicy Bypass -File .\APPCFG-Remediation.ps1 -PolicyName "APPCFG - Ariba - iOS - Pilot" -GroupName "APPCFG - Ariba - Pilot" Apply (explicit): powershell -ExecutionPolicy Bypass -File .\APPCFG-Remediation.ps1 -PolicyName "APPCFG - Ariba - iOS - Pilot" -GroupName "APPCFG - Ariba - Pilot" -Commit Requires the Microsoft Graph SDK and DeviceManagementApps.ReadWrite.All (write only used with -Commit). EXPECTED RESULTS Preview: prints the assignment that would be added. -Commit: adds it and confirms. Existing assignments are preserved (the full set is re-sent). .NOTES Intune Solutions — guarded write (-WhatIf by default; writes only with -Commit). Cloud-aware. No credentials stored. .EXAMPLE .\APPCFG-Remediation.ps1 -PolicyName "APPCFG - Ariba - iOS - Pilot" -GroupName "APPCFG - Ariba - Pilot" -WhatIf > [CmdletBinding(SupportsShouldProcess, ConfirmImpact = 'High')] param([Parameter(Mandatory)][string]$PolicyName, [Parameter(Mandatory)][string]$GroupName, [switch]$Commit, [ValidateSet('Global', 'USGov', 'USGovDoD')] [string]$Environment = 'Global')

APPCFG-Report-AppConfig.ps1

Purpose: Coverage report. Lists the managed apps in the tenant (iOS store/VPP and Android managed Google Play apps) and cross-references the managed-device App Configuration Policies to show, per app, whether an App Configuration Policy is attached. Surfaces the gaps — apps that are deployed but not configured. Read-only. HOW TO RUN powershell -ExecutionPolicy Bypass -File .\APPCFG-Report-AppConfig.ps1 -CsvPath C:\Temp\appcfg-coverage.csv powershell -ExecutionPolicy Bypass -File .\APPCFG-Report-AppConfig.ps1 -Environment USGov Requires the Microsoft Graph SDK and DeviceManagementApps.Read.All. EXPECTED RESULTS One row per app (App / Platform / BundleOrPackage / HasConfig / PolicyName). PASS = every app that should be configured shows HasConfig = True. .NOTES Intune Solutions — read-only. Cloud-aware (Commercial / USGov). No credentials stored. .EXAMPLE .\APPCFG-Report-AppConfig.ps1 -CsvPath C:\Temp\appcfg-coverage.csv

> [CmdletBinding()] param([string]$CsvPath, [ValidateSet('Global', 'USGov', 'USGovDoD')] [string]$Environment = 'Global')

How to run: powershell -ExecutionPolicy Bypass -File .\APPCFG-Report-AppConfig.ps1 -CsvPath C:\Temp\appcfg-coverage.csv powershell -ExecutionPolicy Bypass -File .\APPCFG-Report-AppConfig.ps1 -Environment USGov Requires the Microsoft Graph SDK and DeviceManagementApps.Read.All. EXPECTED RESULTS One row per app (App / Platform / BundleOrPackage / HasConfig / PolicyName). PASS = every app that should be configured shows HasConfig = True. .NOTES Intune Solutions — read-only. Cloud-aware (Commercial / USGov). No credentials stored. .EXAMPLE .\APPCFG-Report-AppConfig.ps1 -CsvPath C:\Temp\appcfg-coverage.csv > [CmdletBinding()] param([string]$CsvPath, [ValidateSet('Global', 'USGov', 'USGovDoD')] [string]$Environment = 'Global')

APPCFG-Validate-Assignment.ps1

Purpose: Answers "did this policy reach this device (or user)?" for a named managed-device App Configuration Policy. Resolves the policy by name, lists its assigned groups, resolves a device (by device name) or user (by UPN) and its group memberships, and reports whether the target is covered by the policy's assignments. Read-only; makes no changes. HOW TO RUN powershell -ExecutionPolicy Bypass -File .\APPCFG-Validate-Assignment.ps1 -PolicyName "APPCFG - Ariba - iOS - Pilot" -DeviceName "IPAD-PILOT-01" powershell -ExecutionPolicy Bypass -File .\APPCFG-Validate-Assignment.ps1 -PolicyName "APPCFG - Outlook - MAM - Pilot" -Upn user@contoso.com -Environment USGov Requires the Microsoft Graph SDK and DeviceManagementApps.Read.All, Directory.Read.All, DeviceManagementManagedDevices.Read.All. EXPECTED RESULTS The policy's assigned groups and a PASS/FAIL: PASS = the device/user is in an assigned group. .NOTES Intune Solutions — read-only. Cloud-aware (Commercial / USGov). No credentials stored. .EXAMPLE .\APPCFG-Validate-Assignment.ps1 -PolicyName "APPCFG - Ariba - iOS - Pilot" -DeviceName "IPAD-PILOT-01" > [CmdletBinding()] param([Parameter(Mandatory)][string]$PolicyName, [string]$DeviceName, [string]$Upn, [ValidateSet('Global', 'USGov', 'USGovDoD')] [string]$Environment = 'Global')

How to run: powershell -ExecutionPolicy Bypass -File .\APPCFG-Validate-Assignment.ps1 -PolicyName "APPCFG - Ariba - iOS - Pilot" -DeviceName "IPAD-PILOT-01" powershell -ExecutionPolicy Bypass -File .\APPCFG-Validate-Assignment.ps1 -PolicyName "APPCFG - Outlook - MAM - Pilot" -Upn user@contoso.com -Environment USGov Requires the Microsoft Graph SDK and DeviceManagementApps.Read.All, Directory.Read.All, DeviceManagementManagedDevices.Read.All. EXPECTED RESULTS The policy's assigned groups and a PASS/FAIL: PASS = the device/user is in an assigned group. .NOTES Intune Solutions — read-only. Cloud-aware (Commercial / USGov). No credentials stored. .EXAMPLE .\APPCFG-Validate-Assignment.ps1 -PolicyName "APPCFG - Ariba - iOS - Pilot" -DeviceName "IPAD-PILOT-01" > [CmdletBinding()] param([Parameter(Mandatory)][string]$PolicyName, [string]$DeviceName, [string]$Upn, [ValidateSet('Global', 'USGov', 'USGovDoD')] [string]$Environment = 'Global')